Process Description and Control

Chapter 3

Concepts

- Computer platform(平台) consists of a collection of hardware resources
- Computer application programs(应用程序) are developed to perform some task
- Operating system manages execution of applications
- Operating system provides a convenient to use, feature rich, secure, and consistent interface (接口) for applications to use
- Operating system provides a uniform, abstract representation of resources that can be requested and accessed by application

Process(进程)

- A program in execution
- An instance of a program running on a computer
- The entity(实体) that can be assigned to and executed on a processor
- A unit of activity characterized by the execution of a sequence(序列) of instructions, a current state(当前状态), and an associated set of system instructions

Process Elements

- Identifier
- State
- Priority
- Program counter
- Memory pointers
- Context data (上下文数据)
- I/O status information
- Accounting information

Process Control Block(PCB)

- Contains the process elements
- Created and managed by the operating system
- Allows support for multiple processes
- Process Table Entry (进程表表项-进程表项)

Process Control Block

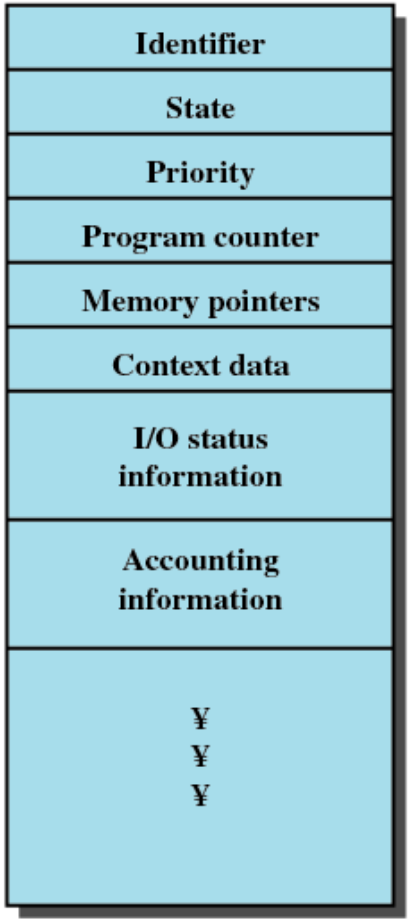


Figure 3.1 Simplified Process Control Block

Trace of Processes (1)

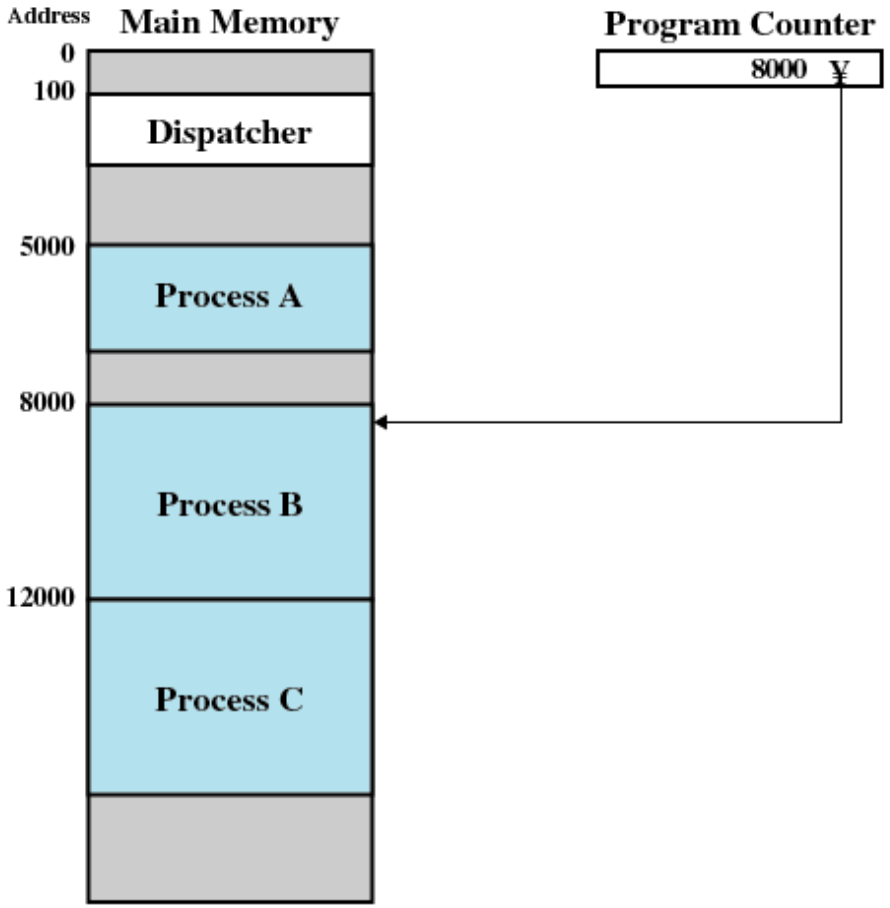


Figure 3.2 Snapshot of Example Execution (Figure 3.4) at Instruction Cycle 13

Trace of Processes (2)

(a) Trace of Process A	(b) Trace of Process B	(c) Trace of Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

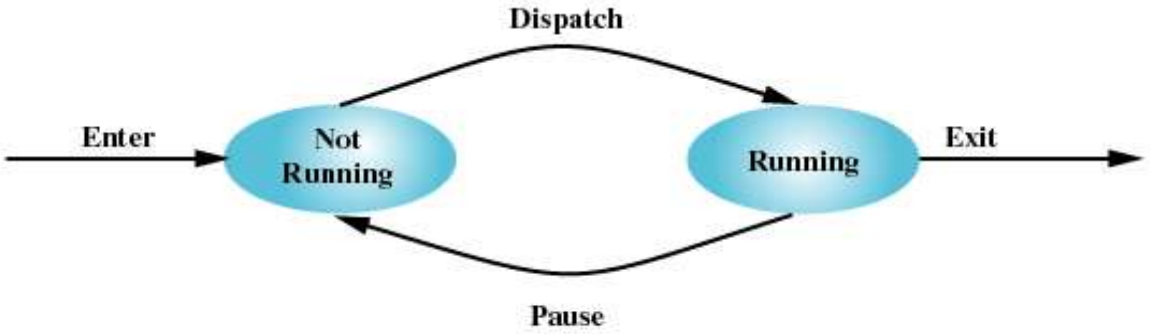
Figure 3.3 Traces of Processes of Figure 3.2

Trace of Processes (3)

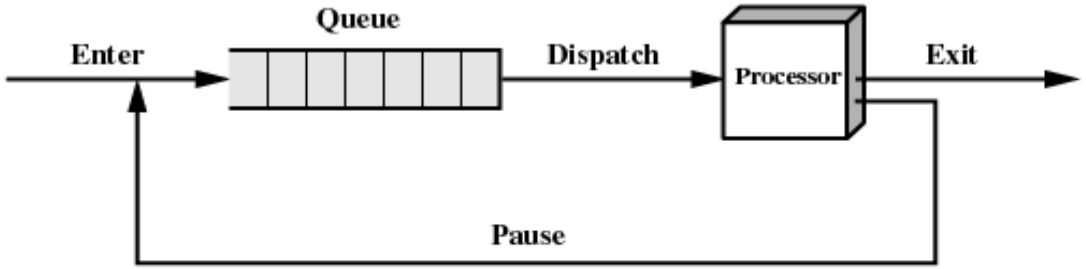
1	5000
2	5001
3	5002
4	5003
5	5004
6	5005
----------	Time out
7	100
8	101
9	102
10	103
11	104
12	105
13	8000
14	8001
15	8002
16	8003
----------	I/O request
17	100
18	101
19	102
20	103
21	104
22	105
23	12000
24	12001
25	12002
26	12003
27	12004
28	12005
----------	Time out
29	100
30	101
31	102
32	103
33	104
34	105
35	5006
36	5007
37	5008
38	5009
39	5010
40	5011
----------	Time out
41	100
42	101
43	102
44	103
45	104
46	105
47	12006
48	12007
49	12008
50	12009
51	12010
52	12011
----------	Time out

Two-State Process Model

- Process may be in one of two states
  - Running
  - Not-running



(a) State transition diagram



(b) Queuing diagram

Process Creation (进程创建)

Table 3.1 Reasons for Process Creation

New batch job	The operating system is provided with a batch job control stream, usually on tape or disk. When the operating system is prepared to take on new work, it will read the next sequence of job control commands.
Interactive logon	A user at a terminal logs on to the system.
Created by OS to provide a service	The operating system can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Process Termination (进程终止)

Table 3.2 Reasons for Process Termination

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation, such as division by zero, or tries to store numbers larger than the hardware can accommodate.

Process Termination (cont.)

Table 3.2 Reasons for Process Termination

Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (for example, if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Differentiation of Not-running

- Not-running
  - ready to execute
- Blocked(阻塞)
  - waiting for I/O
- Dispatcher cannot just select the process that has been in the queue the longest because it may be blocked

A Five-State Model

- Running
- Ready
- Blocked
- New
- Exit

Five-State Process Model

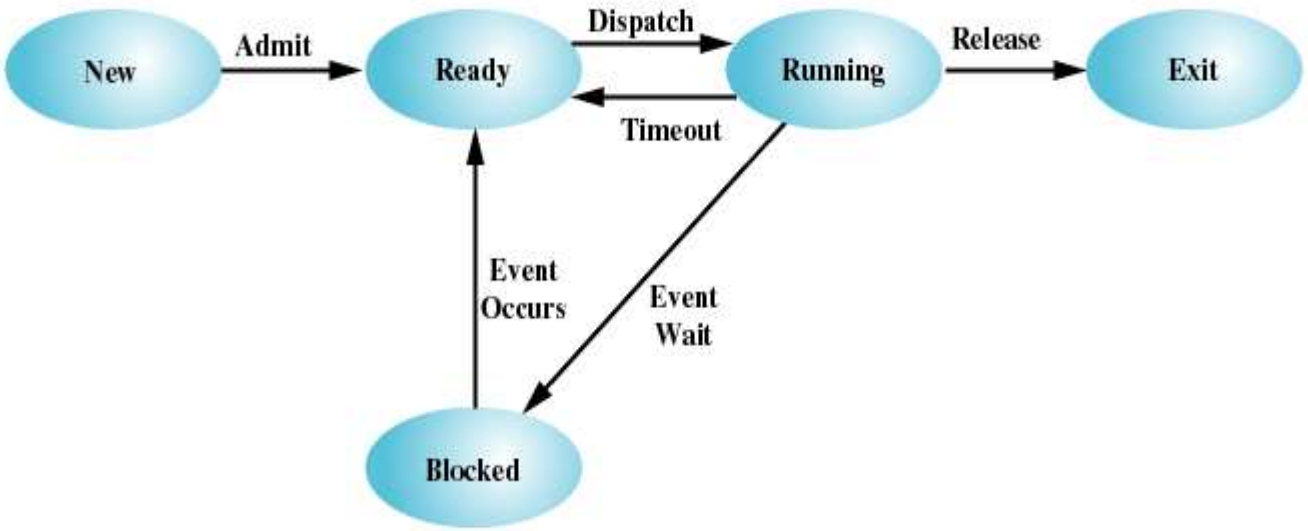


Figure 3.6 Five-State Process Model

States of the Three Processes

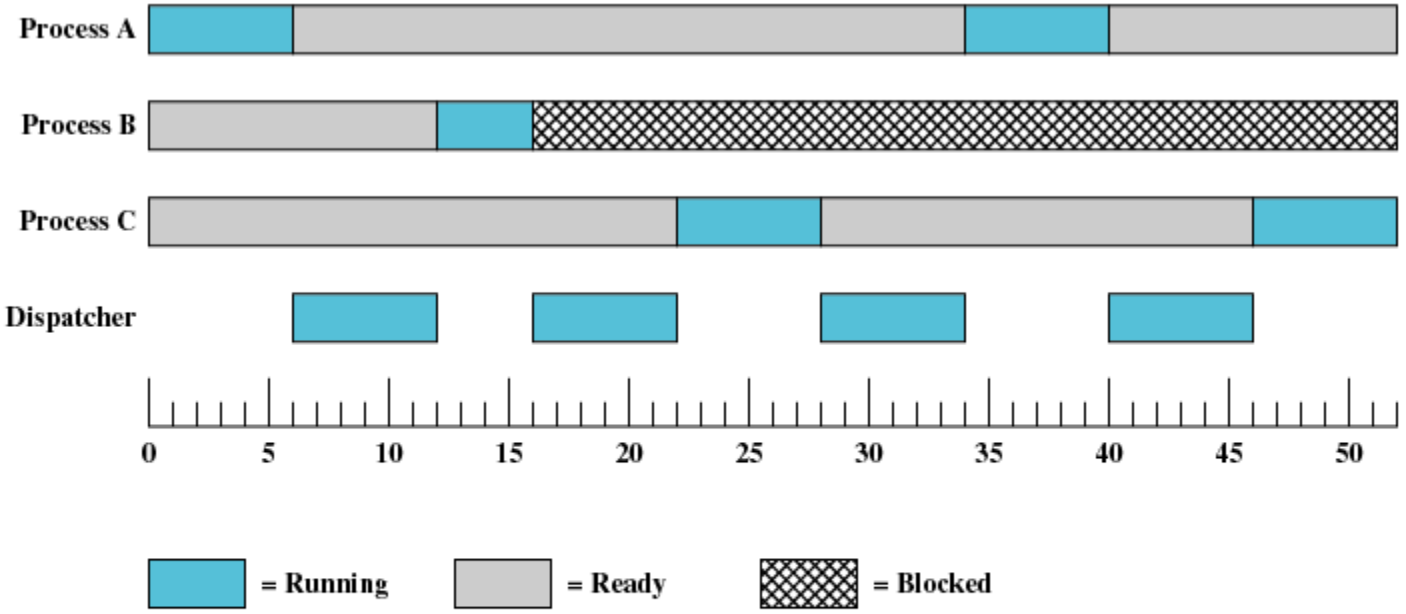
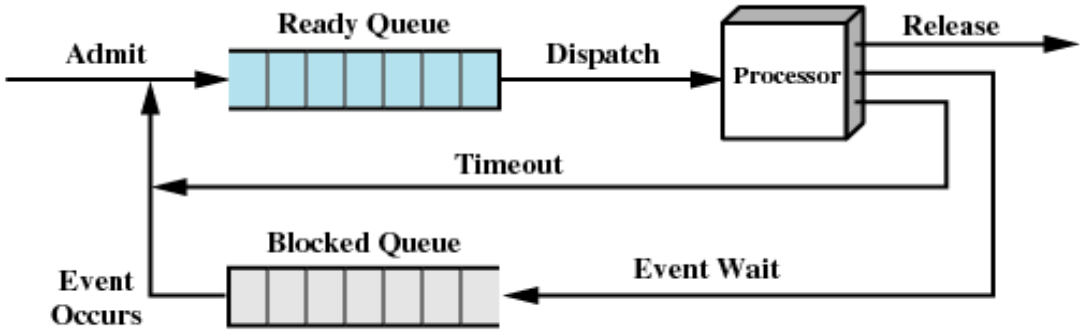


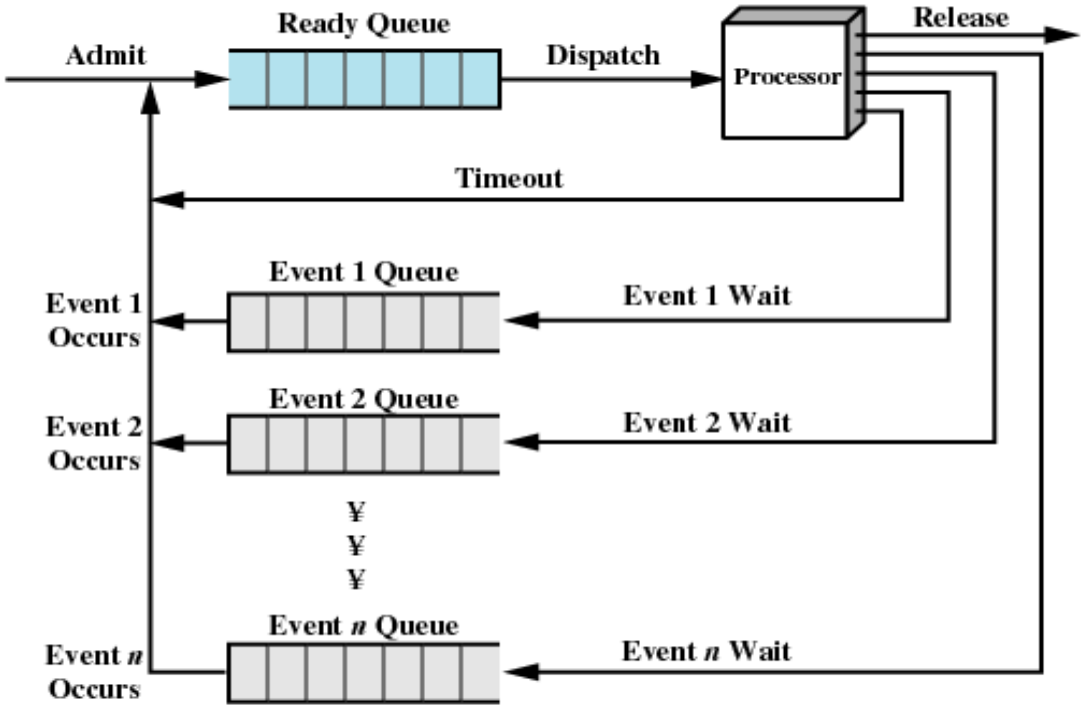
Figure 3.7 Process States for Trace of Figure 3.4

Queues to Implement Five-State Process Model



(a) Single blocked queue

Queues to Implement Five-State Process Model



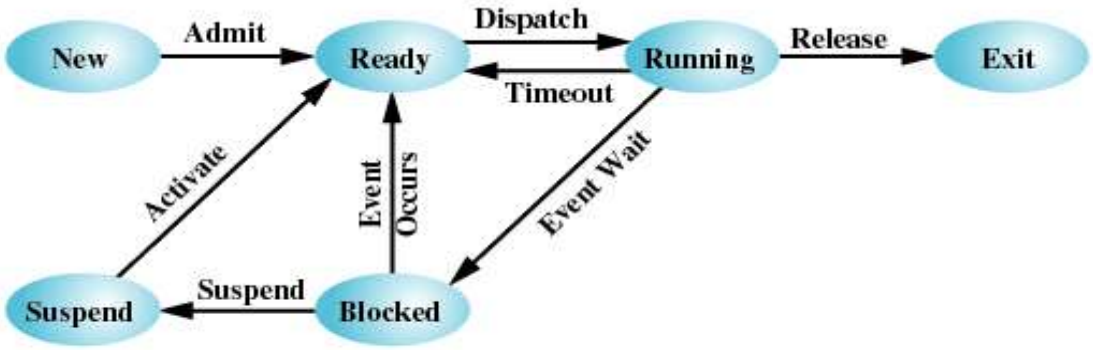
(b) Multiple blocked queues

Figure 3.8 Queuing Model for Figure 3.6

Process Swap Out/Swap In(换出/换入)

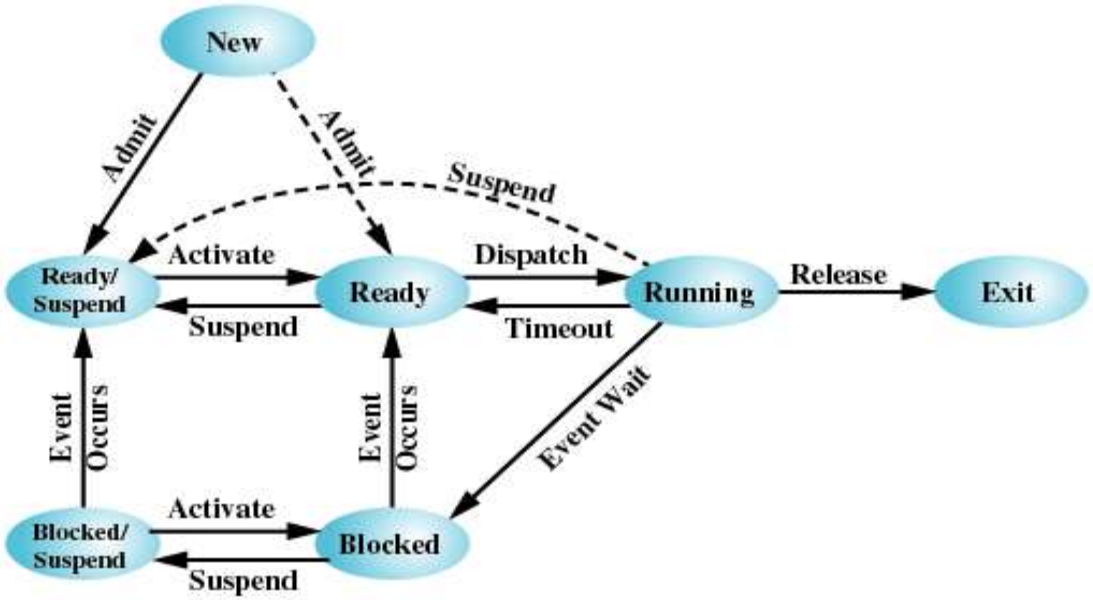
- Processor is faster than I/O so all processes could be waiting for I/O
- Swap these processes to disk to free up more memory
- Blocked state becomes suspend state(换出) when swapped to disk
- Two new states
  - Blocked/Suspend(阻塞/换出)
  - Ready/Suspend(就绪/换出)

One Suspend State



(a) With One Suspend State

Two Suspend States



(b) With Two Suspend States

Figure 3.9 Process State Transition Diagram with Suspend States

Reasons for Process Suspension

Table 3.3 Reasons for Process Suspension

Swapping	The operating system needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The operating system may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendents.

Processes and Resources(进程和资源)

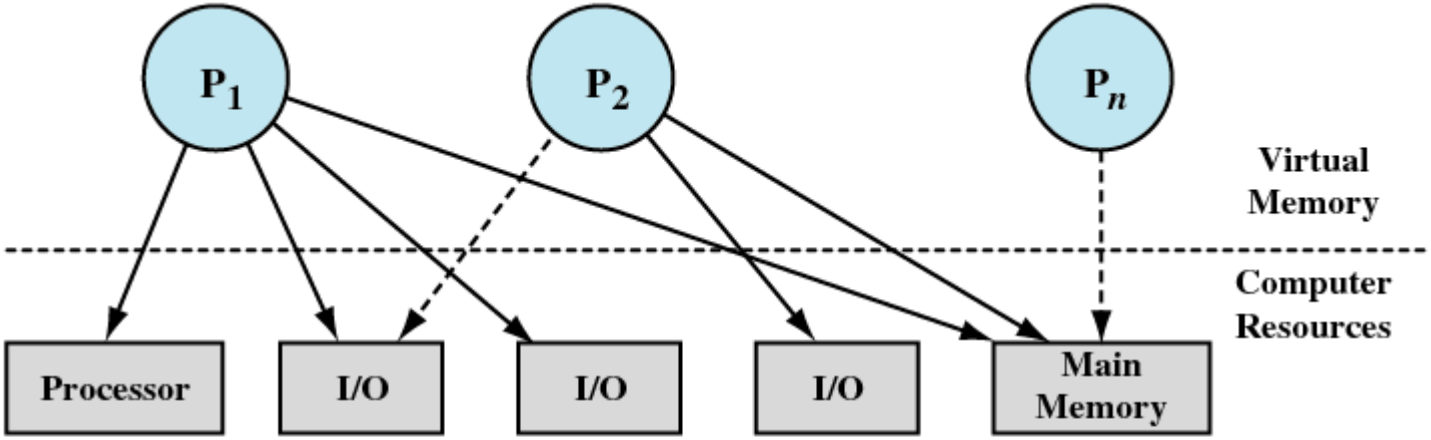


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

Process Control Structures

- Information about the current status of each process and resource
- Operating system manages the tables that are constructed for each entity

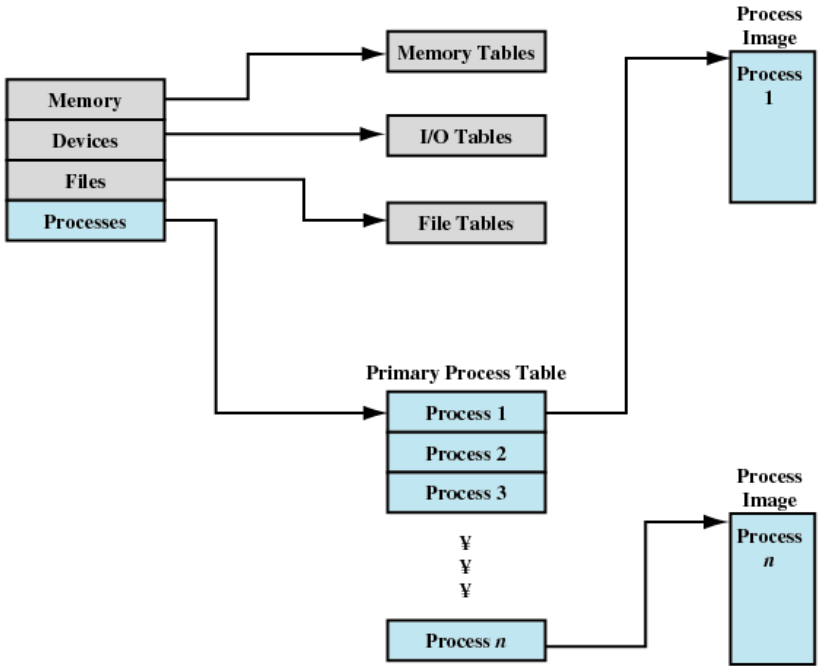


Figure 3.11 General Structure of Operating System Control Tables

Process Control Structures

- Memory Tables
  - Allocation of main memory to processes
  - Allocation of secondary memory to processes
  - Protection attributes for access to shared memory regions
  - Information needed to manage virtual memory

Process Control Structures

- I/O Tables
  - I/O device is available or assigned
  - Status of I/O operation
  - Location in main memory being used as the source or destination of the I/O transfer

Process Control Structures

- File Tables
  - Existence of files
  - Location on secondary memory
  - Current Status
  - Attributes
  - Sometimes this information is maintained by a file management system

Process Control Structures

- Process Table(进程表)
  - Where process is located
  - Attributes in the process control block
    - Program(Code/Text)
    - Data
    - Stack

Process Image (进程映像)

Table 3.4 Typical Elements of a Process Image

User Data The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.
User Program The program to be executed.
System Stack Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.
Process Control Block Data needed by the operating system to control the process (see Table 3.5).

Process Control Block

- Process identification
  - Identifiers
  - Numeric identifiers that may be stored with the process control block include
    - Identifier of this process
    - Identifier of the process that created this process (parent process)
    - User identifier

Process Control Block (cont.)

- Processor State Information
  - User-Visible Registers
    - A user-visible register is one that may be referenced by means of the machine language that the processor executes while in user mode. Typically, there are from 8 to 32 of these registers, although some RISC implementations have over 100.

Process Control Block (cont.)

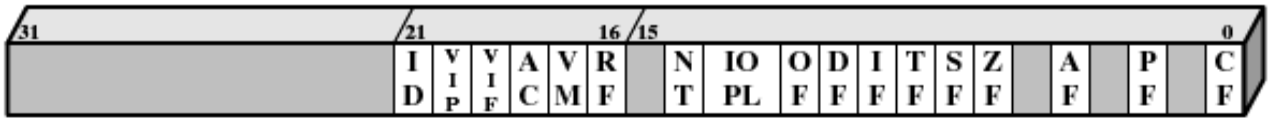
- Processor State Information
  - Control and Status Registers

    These are a variety of processor registers that are employed to control the operation of the processor. These include

    - *Program counter:* Contains the address of the next instruction to be fetched
    - *Condition codes:* Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow)
    - *Status information:* Includes interrupt enabled/disabled flags, execution mode

Process Control Block (cont.)

- Processor State Information



ID = Identification flag
VIP = Virtual interrupt pending
VIF = Virtual interrupt flag
AC = Alignment check
VM = Virtual 8086 mode
RF = Resume flag
NT = Nested task flag
IOPL = I/O privilege level
OF = Overflow flag

DF = Direction flag
IF = Interrupt enable flag
TF = Trap flag
SF = Sign flag
ZF = Zero flag
AF = Auxiliary carry flag
PF = Parity flag
CF = Carry flag

Figure 3.12 Pentium II EFLAGS Register

Process Control Block (cont.)

- Processor State Information
  - Stack Pointers
    - Each process has one or more last-in-first-out (LIFO) system stacks associated with it
    - A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.

Process Control Block (cont.)

- Process Control Information
  - Scheduling and State Information
    - *Process state:* defines the readiness of the process to be scheduled for execution (e.g., running, ready, waiting, halted).
    - *Priority:* One or more fields may be used to describe the scheduling priority of the process. In some systems, several values are required (e.g., default, current, highest-allowable)
    - *Scheduling-related information:* This will depend on the scheduling algorithm used. Examples are the amount of time that the process has been waiting and the amount of time that the process executed the last time it was running.
    - *Event:* Identity of event the process is awaiting before it can be resumed

Process Control Block (cont.)

- Process Control Information
  - Data Structuring
    - A process may be linked to other process in a queue, ring, or some other structure.
    - For example, all processes in a waiting state for a particular priority level may be linked in a queue. A process may exhibit a parent-child (creator-created) relationship with another process. The process control block may contain pointers to other processes to support these structures.

Process Control Block (cont.)

- Process Control Information
  - Interprocess Communication
    - Various flags, signals, and messages may be associated with communication between two independent processes. Some or all of this information may be maintained in the process control block.
  - Process Privileges
    - Processes are granted privileges in terms of the memory that may be accessed and the types of instructions that may be executed. In addition, privileges may apply to the use of system utilities and services.

Process Control Block (cont.)

- Process Control Information
  - Memory Management
    - This section may include pointers to segment and/or page tables that describe the virtual memory assigned to this process.
  - Resource Ownership and Utilization
    - Resources controlled by the process may be indicated, such as opened files. A history of utilization of the processor or other resources may also be included; this information may be needed by the scheduler.

Process Creation

- Assign a unique process identifier
- Allocate space for the process
- Initialize process control block
- Set up appropriate linkages
  - Ex: add new process to linked list used for scheduling queue
- Create of expand other data structures
  - Ex: maintain an accounting file

When to Switch a Process

- Clock interrupt
  - process has executed for the maximum allowable time slice
- I/O interrupt
- Memory fault
  - memory address is in virtual memory so it must be brought into main memory
- Trap
  - error or exception occurred, may cause process to be moved to Exit state
- Supervisor call/system call
  - such as file open

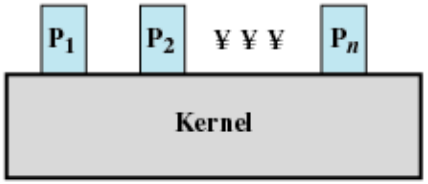
How to Switch a Process

- Save context of processor including program counter and other registers
- Update the process control block of the process that is currently in the Running state
- Move process control block to appropriate queue – ready; blocked;
- Select another process for execution
- Update the process control block of the process selected
- Update memory-management data structures
- Restore context of the selected process

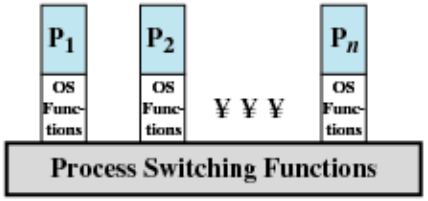
Execution of the Operating System

- Non-process Kernel
  - Execute kernel outside of any process
  - Operating system code is executed as a separate entity that operates in privileged mode
- Execution Within User Processes
  - Operating system software within context of a user process
  - Process executes in privileged mode when executing operating system code
- Process-Based Operating System
  - Implement operating system as a collection of system processes
  - Useful in multi-processor or multi-computer environment

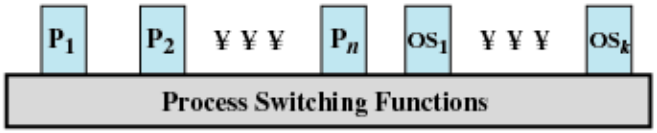
Relationship Between OS and User Processes



(a) Separate kernel



(b) OS functions execute within user processes



(c) OS functions execute as separate processes

Figure 3.15 Relationship Between Operating System and User Processes

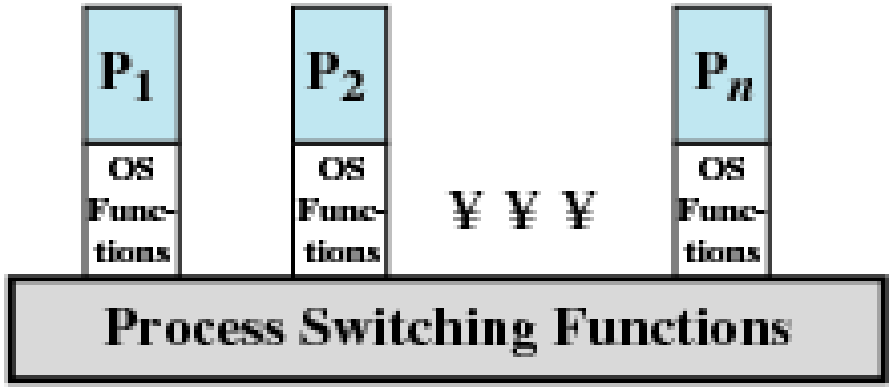
UNIX SVR4 Process Image

Table 3.10 UNIX Process Image

User-Level Context	
Process Text	Executable machine instructions of the program
Process Data	Data accessible by the program of this process
User Stack	Contains the arguments, local variables, and pointers for functions executing in user mode
Shared Memory	Memory shared with other processes, used for interprocess communication
Register Context	
Program Counter	Address of next instruction to be executed; may be in kernel or user memory space of this process
Processor Status Register	Contains the hardware status at the time of preemption; contents and format are hardware dependent
Stack Pointer	Points to the top of the kernel or user stack, depending on the mode of operation at the time or preemption
General-Purpose Registers	Hardware dependent
System-Level Context	
Process Table Entry	Defines state of a process; this information is always accessible to the operating system
U (user) Area	Process control information that needs to be accessed only in the context of the process
Per Process Region Table	Defines the mapping from virtual to physical addresses; also contains a permission field that indicates the type of access allowed the process: read-only, read-write, or read-execute
Kernel Stack	Contains the stack frame of kernel procedures as the process executes in kernel mode

UNIX SVR4 Process Management

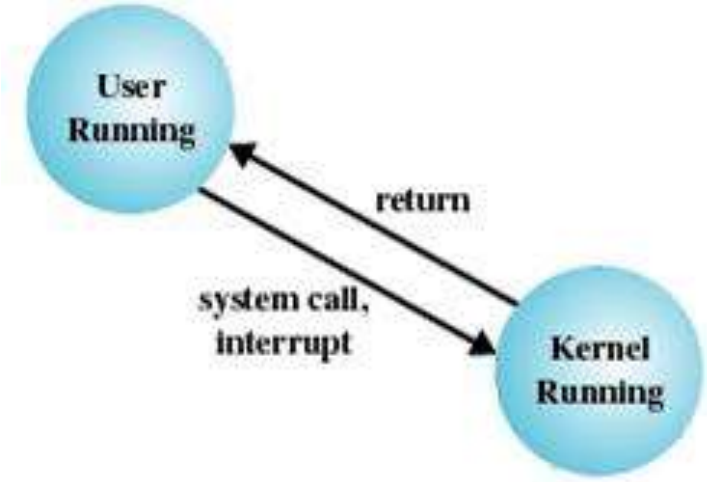
- The operating system executes within the environment of a user process



(b) OS functions execute within user processes

Two Modes of UNIX Process Execution

- User mode (用户态)
  - Less-privileged mode
  - User programs typically execute in this mode
- kernel mode(核心态)
  - More-privileged mode
  - Execution of the operating system



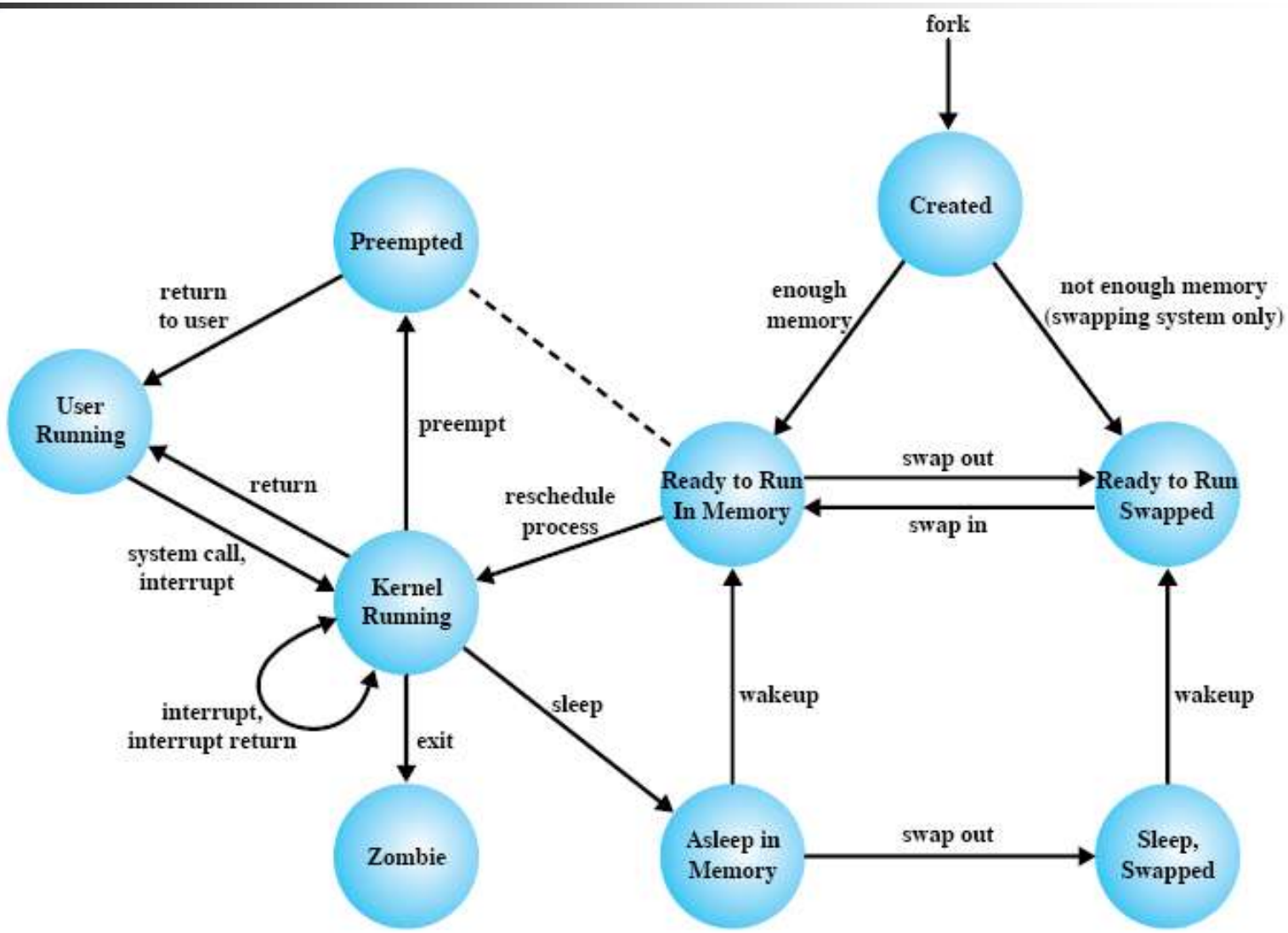
UNIX SVR4 Process States

Process State	Description
User Running	Executing in user mode
Kernel Running	Executing in kernel mode
Ready to Run, in Memory (内存中就绪)	Ready to run as soon as the kernel schedules it
Asleep in Memory (内存中睡眠)	Unable to execute until an event occurs; process is in main memory
Ready to Run, Swapped (磁盘上就绪)	Process is ready to run, but the swapper must swap the process into main memory before the kernel can schedule it to execute

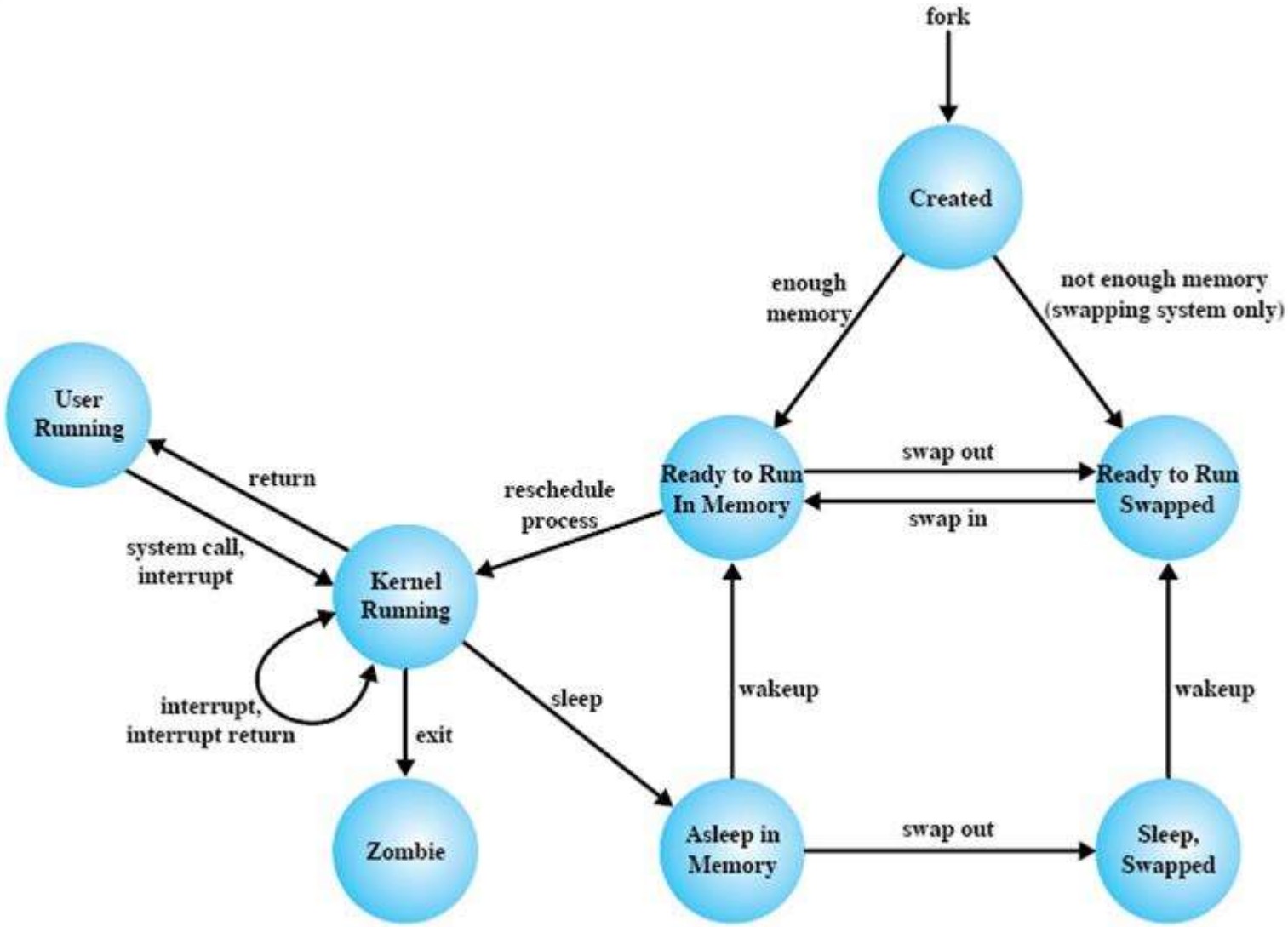
UNIX SVR4 Process States (cont.)

Process State	Description
Sleeping, swapped (磁盘上睡眠)	The process is awaiting an event and has been swapped to secondary storage
Preempted (被抢占)	Process is returning from kernel to user mode, but the kernel preempts (抢占) it and does a process switch to schedule another process
Created	Process is newly created and not yet ready to run
Zombie(僵尸)	Process no longer exists, but it leaves a record for its parent process to collect

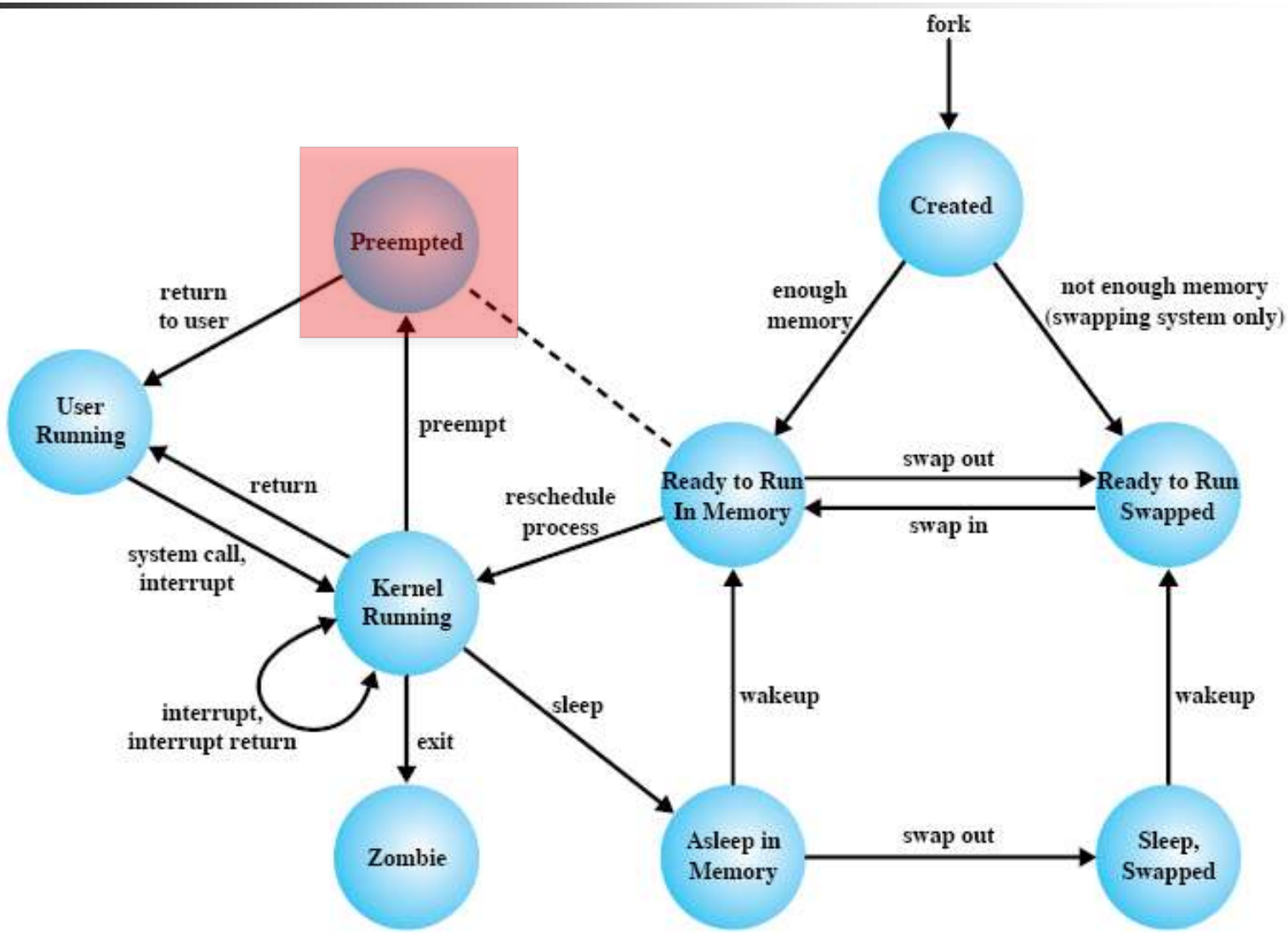
Unix SVR4 Process State Transition



Unix SVR4 Process State Transition



Preemptive and Preempted



Unix System Call: fork

```
#include <sys/types.h>
#include <unistd.h>

void main()
{
    int x = 0;
    fork();
    x = 1;
    return;
}
```

Unix System Call: fork

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main()
{
    pid_t pid;
    if ((pid = fork()) == 0) {
       fprintf(stderr, "I am the child, id = %ld\n", (long)getpid());
    }
    else {
       fprintf(stderr, "I am the parent, id = %ld\n", (long)getpid());
    };
    return;
}
```

Unix System Call: fork

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main()
{
    pid_t pid;
    for (int i = 1; i < 4; i++)
       if (pid = fork())  break;

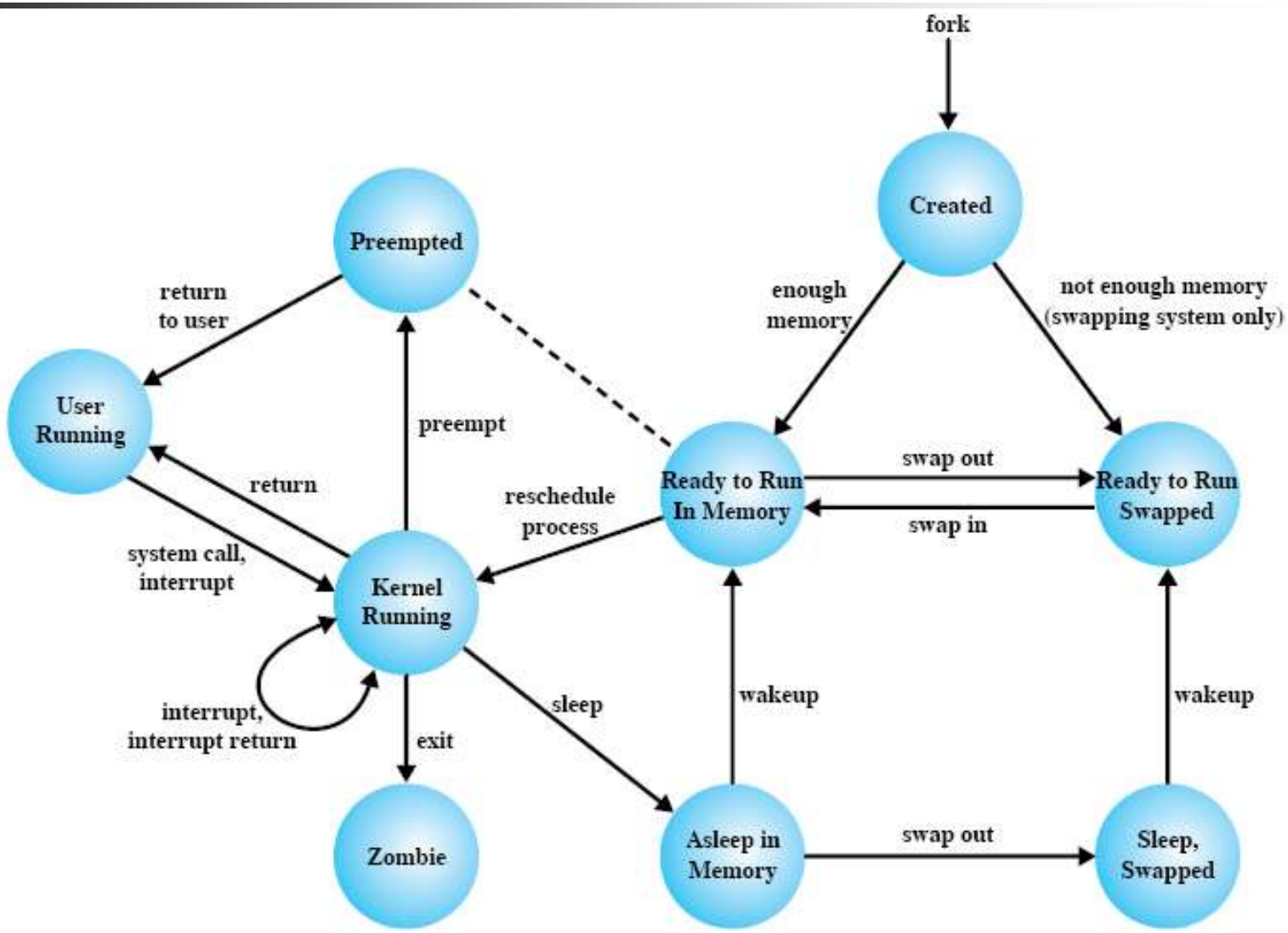
     fprintf(stderr, "This is process %ld with parent %ld\n",
    (long)getpid(), (long)getppid());

    return;
}
```

Unix Shell Skeleton (代码框架)

```
while(TRUE){
    read-command(command, parameters);
    if (fork() != 0) {
        /* parent code */
        waitpid(-1, &status, 0);
    }
    else {
        /* child code */
        execve(command, parameters, 0);
    }
}
```

SVR4 Process State Transition Review



Zombie: mytest.c

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int main(int   argc ,   char   *argv[] )
{
   int   fd;
   if (argc<2) {
      printf( ”Arguments too few!\n” );
      exit(1);
   }
   fd = open(argv[1],  …);
   …
   close(fd);
   return  0;
}
```

mytest.c compilation and running

```
$gcc mytest.c –o mytest
$./mytest
Arguments too few!
$
$./mytest readme.txt
……
$
```